SOF 103
C AND C++ PROGRAMMING

Lecture 2
Variables and Data Types

LAST WEEK REVIEW

The C++ program consist of four parts:

1. **The pre-processor directive.**
2. **The namespace.**
3. **The main function block.**
4. **The program statements.**

```
# include <iostream>
using namespace std;

int main ( )
{
      cout  << “Welcome to C++” ;
      return 0;
}
```

CHAPTER OVERVIEW

- VARIABLES, CONSTANT AND ASSIGNMENT STATEMENTS
- DATA TYPES AND EXPRESSIONS
- ARITHMETIC OPERATORS
- INPUT AND OUTPUT
- OPERATORS PRECEDENCE

VARIABLES DECLARATION

Variable Declarations

All variables must be declared before they are used. The syntax for variable declarations is as follows:

SYNTAX

```
Type_Name Variable_Name_1, Variable_Name_2, ...;
```

EXAMPLES

```
int count, number_of_dragons, number_of_trolls;
double distance;
```

VARIABLES DECLARATION

- Example of variables declaration:

```
#include <iostream>
using namespace std;
// Calculate and output product of 2 numbers
int main( )
{
    int number1, number2;
    float product;

    number1 = 25;
    number2 = 4;

    product = number1 * number2;
    cout << "The product = " << product;
}
```

VARIABLES

- C++ variable can **hold a number or data of other types**.
- The number or other type of data held in a variable is called its **value**.
- Once a variable is declared, the system allocates a memory space for it, then we may store a value in that space.

- Each variable must have a **data type**.
  - It tells the system how much memory space to be allocated.
  - It tells the system how to interpret the 0s and 1s

IDENTIFIERS

- Variables can also be referred to as identifiers.
- Always use a meaningful name for the variable.
  - "incomeTax" or "income_Tax" instead of just "Tax" or "T"
  - "number_of_students" instead of "number"
  - "sales" instead of "s"

VARIABLES

Rules for Variable name:

- **Variable names in C++ can range from 1 to 255 characters.**
- **All variable names must begin with a letter of the alphabet or an underscore(_).**
- **After the first initial letter, variable names can also contain letters and numbers.**
- **Variable names are case sensitive. ("Num" ≠ "num")**
- **No spaces or special characters are allowed.**
- **You cannot use a C++ keyword (a reserved word) as a variable name.**

VALID AND INVALID VARIABLES

IDENTIFIER	VALID?	REASON IF INVALID
totalSales	YES	
total_Sales	YES	
total.Sales	NO	Cannot contain .
4thQtrSales	NO	Cannot begin with a digit
totalSale$	NO	Cannot contain $

VARIABLES

Determine whether the following variable names are valid:

1. Number_5 **Valid**
2. _number5 **Valid**
3. number$123 **Invalid**
4. number 7 **Invalid**
5. 7number **Invalid**
6. Number-10 **Invalid**
7. 123number10 **Invalid**
8. main **Invalid**
9. main4 **Valid**
10. total income **Invalid**
11. total_income **Valid**
12. totalincome **Valid**
13. Total_income **Valid**
14. string **Invalid**

C++ KEYWORDS

You cannot use any of the C++ key words below as an identifier.

These words have reserved meaning.

Table 2-4 The C++ Key Words

asm	auto	break	bool	case
catch	char	class	const	const_cast
continue	default	delete	do	double
dynamic_cast	else	enum	explicit	extern
false	float	for	friend	goto
if	inline	int	long	mutable
namespace	new	operator	private	protected
public	register	reinterpret_cast	return	short
signed	sizeof	static	static_cast	struct
switch	template	this	throw	true
try	typedef	typeid	typename	union
unsigned	using	virtual	void	volatile
wchar_t	while			

LITERALS

- A **literal** is **any notation for representing a value within source code**.
- For examples,

```
number = 10;                (10 is an integer literal)
                            (number is a variable)
cout << “Welcome”;          (Welcome is a string literal)
answer = True;              (True is a boolean literal)
                            (answer is a variable)
```

DECLARATION AND INITIALIZATION

- A variable can be declared with or without initialization.

1. `int myNumber = 100;`
2. `int myNumber;`
   `myNumber = 100;`

Address	Identifier	Value
0x12ed2D	myNumber	100

Which are the variables and which are the literal?

```
int main()
{
    int number;

    number = 5;
    cout << "The value of number is " << "number" << endl;
    cout << "The value of number is " << number << endl;

    number = 7;
    cout << "Now the value of number is " << number << endl;

    return 0;
}
```

This is a variable

This is a string literal

5 is an integer literal

This is a string literal

This is a variable

Which are the output of the following program?

```
int main()
{
    int number;

    number = 5;
    cout << "The value of number is " << "number" << endl;
    cout << "The value of number is " <<  number  << endl;

    number = 7;
    cout << "Now the value of number is " << number << endl;

    return 0;
}
```

The value of number is number
The value of number is 5
Now the value of number is 7

CONSTANTS

- Unlike a variable, a **constant** is a data item whose value cannot change/alter during the program's execution.
- Two different ways to declare constant, using the word **const** or **#define**.
- For example,

```
const float PI = 3.1415927;
const int number_of_day_in_a_week = 7;
#define PI 3.1415927
#define minutesPerHour 60
```

CONSTANTS

- **Try to change the value of PI below will result in error.**

```
# include <iostream>
using namespace std;
#define days_in_a_week   7
int main ( )   {
      const float PI = 3.1415927;

      float radius=5, area_of_circle;
      area_of_circle = 2 * PI * radius;
      PI = 8.5;  // Error
      area_of_circle = 2 * PI * radius;
      return 0;
}
```

ASSIGNMENT STATEMENT

- **Syntax:**

  Variable = Expression;

- **Examples**
  - distance = rate * time;
  - count = count + 2;
  - speed = velocity;

The expression on the right-hand side of the equal sign is evaluated first, and the result is then assigned to the variable on the left-hand side.

C++ DATA TYPES

- Data is information that can be stored and used by a computer program.
- There are many different types of data.
- Variables are classified according to their data type, which determines the kind of information that may be stored in them.

C++ DATA TYPES

There are 10 basic/primitive data types in C++.

Category	Data Type	Bytes
Integers	bool	1
	char	1
	int	4
	short	2
	long	4
	Unsigned int	4
	Unsigned short	2
	Unsigned long	4
Fractional numbers	float	4
	double	8

INTEGER DATA TYPES SIZES & RANGES

Data Type	Size	Range
short	2 bytes	–32,768 to +32,767
unsigned short	2 bytes	0 to +65,535
int	4 bytes	–2,147,483,648 to +2,147,483,647
unsigned int	4 bytes	0 to 4,294,967,295
long	4 bytes	–2,147,483,648 to +2,147,483,647
unsigned long	4 bytes	0 to 4,294,967,295

- The number of bytes is compiler dependent. Depending on your operating system, the sizes and ranges may be different.
- E.g. int 2 bytes for 16-bit systems (MS DOS)

SIGNED DATA TYPES

Keyword	Numerical Range Low	High	Digits of Precision	Bytes of Memory
char	–128	127	n/a	1
short	–32,768	32,767	n/a	2
int	–2,147,483,648	2,147,483,647	n/a	4
long	–2,147,483,648	2,147,483,647	n/a	4
float	3.4 x 10–38	3.4 x 1038	7	4
double	1.7 x 10–308	1.7 x 10308	15	8
long double	3.4 x 10–4932	1.1 x 104932	19	10

UNSIGNED DATA TYPES

Keyword	Numerical Range		Bytes of Memory
	Low	High	
unsigned char	0	255	1
unsigned short	0	65,535	2
unsigned int	0	4,294,967,295	4
unsigned long	0	4,294,967,295	4

THINGS TO CONSIDER WHILE USING NUMERIC DATA

Your primary considerations for selecting the best data type for a numeric variable are the following:

- **Whether the variable needs to hold integers or floating-point values, the largest and smallest numbers that the variable needs to be able to store.**
- **Whether the variable needs to hold signed (both positive and negative) or only unsigned (just zero and positive) numbers.**
- **The number of decimal places of precision needed for values stored in the variable.**

WHAT DATA TYPE IS SUITABLE?

- student_mark — short or unsigned short
- mobile_phone_number — int or unsigned int
- distance_from_the_moon — float or double
- pass_or_fail — char or short
- posterCode — short or unsigned short
- probability — float
- student_height — float

DEFINING VARIABLES

- Variables of the same type can be defined

  - On separate lines:

```
int length;
int width;
float area;
```

  - On the same line:

```
int length, width;
float area;
```

- Variables of different types must be in different definitions

PROGRAM TO DEMONSTRATE "INTEGER VARIABLES"

```
// This program defines three variables in the same statement.
// They are given initial values at the time they are defined.
int main()
{
   int floors =  15,
       rooms  = 300,
       suites =  30;

   cout << "The Grande Hotel has " << floors << " floors\n";
   cout << "with " << rooms << " rooms and " << suites;
   cout << " suites.\n";
   return 0;
}
```

Program Output

```
The Grande Hotel has 15 floors
with 300 rooms and 30 suites.
```

FORMATTING FOR NUMBERS WITH A DECIMAL POINT

- When the computer outputs a value of type *double* or *float* the format may not be what you have expected.
- For example, the output produced by the following statement

```
double price = 78.5;
cout << "The price is $" << price << endl;
```

might be `The price is $78.5`

or might be `The price is $78.500000`

or might even be `The price is $7.850000e01`

USE FIXED AND SETPRECISION TO FORMAT THE DECIMAL

- Use **fixed** and ***setprecision*** to format the decimal output to the number of decimal places that you desire.
- For example, the output produced by the following statement

```
double price = 78.5;
cout << fixed<< setprecision(2);
cout << "The price is $" << price << endl;
```

will be **The price is $78.50**

Must add in #include <iomanip> to make it works.

THE CHAR DATA TYPE

- **A variable of the char data type holds only a single character.**

```
// This program uses character constants.
int main()
{
   char letter;

   letter = 'A';
   cout << letter << endl;

   letter = 'B';
   cout << letter << endl;
   return 0;
}
```

Program Output

```
A
B
```

```
char letterOne = 'A';      // This will work.
char letterTwo = "A";      // This will NOT work!
```

RELATIONSHIP BETWEEN CHARACTERS & INTEGERS

```
// This program demonstrates the close relationship between
// characters and integers.
```

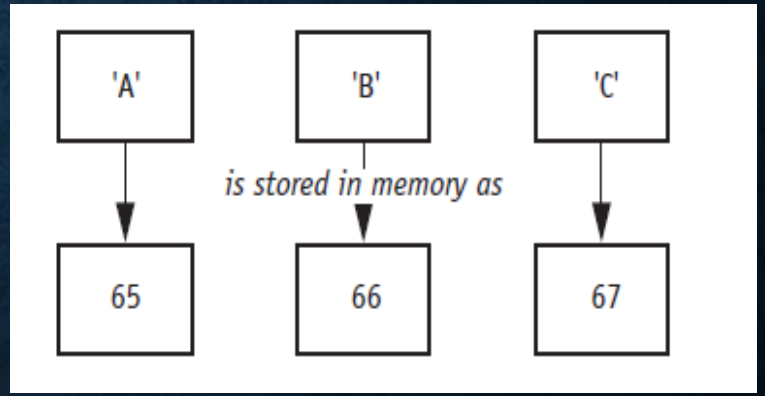
```
int main()
{
   char letter;

   letter = 65;
   cout << letter << endl;

   letter = 66;
   cout << letter << endl;
   return 0;
}
```

Program Output

```
A
B
```



ASCII CODE

0	00	NUL
1	01	SOH
2	02	STX
3	03	ETX
4	04	EOT
5	05	ENQ
6	06	ACK
7	07	BEL
8	08	BS
9	09	HT
10	0A	LF
11	0B	VT
12	0C	FF
13	0D	CR
14	0E	SO
15	0F	SI
16	10	DLE
17	11	DC1
18	12	DC2
19	13	DC3
20	14	DC4
21	15	NAK
22	16	SYN
23	17	ETB
24	18	CAN
25	19	EM
26	1A	SUB
27	1B	ESC
28	1C	FS
29	1D	GS
30	1E	RS
31	1F	US
32	20	space
33	21	!
34	22	"
35	23	#
36	24	$
37	25	%
38	26	&
39	27	'
40	28	(
41	29	)
42	2A	*
43	2B	+
44	2C	,
45	2D	-
46	2E	.
47	2F	/
48	30	0
49	31	1
50	32	2
51	33	3
52	34	4
53	35	5
54	36	6
55	37	7
56	38	8
57	39	9
58	3A	:
59	3B	;
60	3C	<
61	3D	=
62	3E	>
63	3F	?
64	40	@
65	41	A
66	42	B
67	43	C
68	44	D
69	45	E
70	46	F
71	47	G
72	48	H
73	49	I
74	4A	J
75	4B	K
76	4C	L
77	4D	M
78	4E	N
79	4F	O
80	50	P
81	51	Q
82	52	R
83	53	S
84	54	T
85	55	U
86	56	V
87	57	W
88	58	X
89	59	Y
90	5A	Z
91	5B	[
92	5C	\
93	5D	]
94	5E	^
95	5F	_
96	60	`
97	61	a
98	62	b
99	63	c
100	64	d
101	65	e
102	66	f
103	67	g
104	68	h
105	69	i
106	6A	j
107	6B	k
108	6C	l
109	6D	m
110	6E	n
111	6F	o
112	70	p
113	71	q
114	72	r
115	73	s
116	74	t
117	75	u
118	76	v
119	77	w
120	78	x
121	79	y
122	7A	z
123	7B	{
124	7C	|
125	7D	}
126	7E	~
127	7F	DEL

EXTENDED ASCII CODE

128	80	□
129	81	□
130	82	□
131	83	□
132	84	□
133	85	□
134	86	□
135	87	□
136	88	□
137	89	□
138	8A	□
139	8B	□
140	8C	□
141	8D	□
142	8E	□
143	8F	□
144	90	□
145	91	□
146	92	□
147	93	□
148	94	□
149	95	□
150	96	□
151	97	□
152	98	□
153	99	□
154	9A	□
155	9B	□
156	9C	□
157	9D	□
158	9E	□
159	9F	□
160	A0	
161	A1	¡
162	A2	¢
163	A3	£
164	A4	¤
165	A5	¥
166	A6	¦
167	A7	§
168	A8	¨
169	A9	©
170	AA	ª
171	AB	«
172	AC	¬
173	AD	
174	AE	®
175	AF	¯
176	B0	°
177	B1	±
178	B2	²
179	B3	³
180	B4	´
181	B5	µ
182	B6	¶
183	B7	·
184	B8	¸
185	B9	¹
186	BA	º
187	BB	»
188	BC	¼
189	BD	½
190	BE	¾
191	BF	¿
192	C0	À
193	C1	Á
194	C2	Â
195	C3	Ã
196	C4	Ä
197	C5	Å
198	C6	Æ
199	C7	Ç
200	C8	È
201	C9	É
202	CA	Ê
203	CB	Ë
204	CC	Ì
205	CD	Í
206	CE	Î
207	CF	Ï
208	D0	Ð
209	D1	Ñ
210	D2	Ò
211	D3	Ó
212	D4	Ô
213	D5	Õ
214	D6	Ö
215	D7	×
216	D8	Ø
217	D9	Ù
218	DA	Ú
219	DB	Û
220	DC	Ü
221	DD	Ý
222	DE	Þ
223	DF	ß
224	E0	à
225	E1	á
226	E2	â
227	E3	ã
228	E4	ä
229	E5	å
230	E6	æ
231	E7	ç
232	E8	è
233	E9	é
234	EA	ê
235	EB	ë
236	EC	ì
237	ED	í
238	EE	î
239	EF	ï
240	F0	ð
241	F1	ñ
242	F2	ò
243	F3	ó
244	F4	ô
245	F5	õ
246	F6	ö
247	F7	÷
248	F8	ø
249	F9	ù
250	FA	ú
251	FB	û
252	FC	ü
253	FD	ý
254	FE	þ
255	FF	ÿ

CHAR VS STRING DATA TYPE

- The char data type is used to store a single character/letter.
- The character must be surrounded by single quotes.

Example:
```
char myGrade;
myGrade = 'A';
```

- The string type is used to store a sequence of characters (i.e. 1 or more characters).
- String values must be surrounded by double quotes.

Example:
```
string myGrade, grade_in_word;
myGrade = "A";
grade_in_word = "Distinction";
```

DETERMINING THE SIZE OF A DATA TYPE

- The **sizeof** operator may be used to determine the size of a data type on any system.

```
int main()
{
    long double apple;

    cout << "The size of an integer is " << sizeof(int);
    cout << " bytes.\n";
    cout << "The size of a long integer is " << sizeof(long);
    cout << " bytes.\n";
    cout << "An apple can be eaten in " << sizeof(apple);
    cout << " bytes!\n";
    return 0;
}
```

```
The size of integer is 4 bytes
The size of long integer is 4 bytes
The size of long double is 16 bytes

Process returned 0 (0x0)   execution time : 14.053 s
Press any key to continue.
```

FUNDAMENTAL ARITHMETIC OPERATORS

Operator	Name	Description	Example
+	Addition	Adds together two values	x + y
-	Subtraction	Subtracts one value from another	x - y
*	Multiplication	Multiplies two values	x * y
/	Division	Divides one value by another	x / y
%	Modulus	Returns the division remainder	x % y
++	Increment	Increases the value of a variable by 1	++x
--	Decrement	Decreases the value of a variable by 1	--x

EXAMPLES OF ARITHMETIC OPERATORS

```
sum = 10 + 12;                  22
diff = 32 – 48;                 -16
area = 9 * 20;                  180
div1 = 6 / 3;                   2
div2 = 11 / 2;                  5
div3 = 11 / 2.0;                5.5
div4 = 15.0 / 3.0;              5.0
rem = 20 % 6;                   2
rem = 1 % 3;                    1
count=5; count++ or ++count;    6
count=5; count--  or  -- count; 4
```

++ AND -- OPERATORS

What are the output of the following statements?

(a)

```
count = 10;
cout << count++ << endl;      10
cout << ++count << endl;      12
cout << count-- << endl;      12
cout << --count << endl;      10
```

(b)

```
count = 10;
num = 5;
cout << (count++) + (num++) << endl;      15
cout << (count++) + (++num) << endl;      18
cout << (++count) + (--num) << endl;      19
```

SAMPLE PROGRAM

```
// This program is to calculate the area and perimeter of a rectangle
#include <iostream>
using namespace std;

int main( )
{
    int length = 10, width = 8;
    float area, perimeter;

    area = length * width;
    perimeter = 2 * length + 2 * width;

    cout << "Area = " << area << " ; Perimeter  = "<< perimeter << endl;
    return 0;
}
```

How many variables are required?

OUTPUT

Area = 80; Perimeter = 36

THE INPUT STATEMENT USING CIN OBJECT

- cin accepts data input from the console input like keyboard.
- cin takes the value from the user at run time and stores it in the variable.

The syntax:

```
cin >> variableName;
```

- Usually cin works together with the cout to make the program easy to understand during run time.

PROGRAM TO TAKE INPUT FROM USER

```
// This program calculates and displays the area of a rectangle.

int main()
{
   int length, width, area;

   cout << "This program calculates the area of a rectangle.\n";

   // Have the user input the rectangle's length and width
   cout << "What is the length of the rectangle? ";
   cin  >> length;
   cout << "What is the width of the rectangle? ";
   cin  >> width;

   // Compute and display the area
   area = length * width;
   cout << "The area of the rectangle is " << area << endl;
   return 0;
}
```

ENTERING MULTIPLE VALUES

```
// This program calculates and displays the area of a rectangle.
int main()
{
   int length, width, area;

   cout << "This program calculates the area of a rectangle.\n";

   // Have the user input the rectangle's length and width
   cout << "Enter the length and width of the rectangle ";
   cout << "separated by a space.\n";
   cin  >> length >> width;

   // Compute and display the area
   area = length * width;
   cout << "The area of the rectangle is " << area << endl;
   return 0;
}
```

CIN TO READ MULTIPLE VALUES OF DIFFERENT DATA TYPES

```
// This program demonstrates how cin can read multiple values
// of different data types.
```

```
int main()
{
    int whole;
    double fractional;
    char letter;

    cout << "Enter an integer, a double, and a character: ";
    cin  >> whole >> fractional >> letter;

    cout << "whole: " << whole << endl;
    cout << "fractional: " << fractional << endl;
    cout << "letter: " << letter << endl;
    return 0;
}
```

Program Output with Example Input Shown in Bold

```
Enter an integer, a double, and a character: 4 5.7 b[Enter]
whole: 4
fractional: 5.7
letter: b
```

INPUT EXAMPLE USING CIN

```
#include <iostream>
using namespace std;

// This program is to calculate the area and perimeter of a rectangle
int main( )  {
     int length, width;
     float area, perimeter;

     cout << "Enter length and width of a rectangle: ";
     cin >> length >> width

     area = length * width;
     perimeter = 2 * length + 2 * width;

     cout << "Area = " << area << " ; Perimeter = "<< perimeter << endl;
     return 0;
}
```

OPERATOR PRECEDENCE

- **Operator precedence describes the order in which C/C++ evaluates the expression.**

Table 3-1 Precedence of Arithmetic Operators (Highest to Lowest)

( )		Expressions within parentheses are evaluated first
-	unary	Negation of a value, e.g., -6
* / %	binary	Multiplication, division, and modulus
+ -	binary	Addition and subtraction

OPERATOR PRECEDENCE

Expression	Value
5 + 2 * 4	13
10 / 2 - 3	2
8 + 12 * 2 - 4	28
4 + 17 % 2 - 1	4
6 - 3 * 2 + 7 - 1	6

OPERATOR PRECEDENCE

Table 3-4 More Arithmetic Expressions

Expression	Value
(5 + 2) * 4	28
10 / (5 - 3)	5
8 + 12 * (6 - 2)	56
(4 + 17) % 2 - 1	0
(6 - 3) * (2 + 7) / 3	9

ARITHMETIC & ALGEBRAIC EXPRESSIONS

Algebraic Expression	C++ Expression
$y = \frac{3x}{2}$	`y = x / 2 * 3;`
$z = 3bc + 4$	`z = 3 * b * c + 4;`
$a = \frac{3x + 2}{4a - 1}$	`a = (3 * x + 2) / (4 * a - 1)`

ASSIGNMENT STATEMENTS

Operator	Example	Same As
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3
&=	x &= 3	x = x & 3
|=	x |= 3	x = x | 3
^=	x ^= 3	x = x ^ 3
>>=	x >>= 3	x = x >> 3
<<=	x <<= 3	x = x << 3

REFERENCES

1. Problem Solving with C++ (10th Edition), Walter Savitch, Addison Wesley, 2018
2. Paul J. Dietel, C++ How to Program (10th Edition), Pearson, 2017.
3. Object Oriented Programming in C++, Fourth Edition, Robert Lafore.

```cpp
#include <iostream>
using namespace std;

int main()  {
    int choice;

    do {
        cout << "\n Choose 1 to see the next homework assignment."
         << "\n Choose 2 for your grade on the last assignment."
         << "\n Choose 3 for assignment hints."
         << "\n Choose 4 to exit this program.";
        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice)  {
            case 1: // Code to display next assignment.
                break;
            case 2: // Code to display student grade.
                break;
            case 3: // Code to display assignment hints.
                break;
            case 4: cout << "End of Program.";
                break;
            default: cout << "Not a valid choice. Enter your choice again."
    } while (choice != 4)
    return 0;
}
```